SORT IT

INSTITUTIONAL FINANCE INTELLIGENCE PLATFORM

MVP TECHNICAL SPECIFICATION

Product Engineering Document · v1.0 · April 2026

Classification	CONFIDENTIAL
Product	Sort IT — Web SaaS Platform
Target Release	Q3 2026 (10-Week Sprint)
Author	Anish — Founder / Lead Engineer
Stack	Next.js · FastAPI · PostgreSQL · Claude AI

Table of Contents

1. Executive Summary & Vision
2. Product Philosophy & Design System
3. MVP Feature Set (Scope & Non-Scope)
4. Technical Architecture
5. Frontend Engineering Specification
6. Backend Engineering Specification
7. AI / Intelligence Layer
8. Database Schema
9. API Design
10. Security & Compliance
11. UI/UX Design System
12. 10-Week Build Roadmap
13. Infrastructure & DevOps
14. Success Metrics & KPIs

01 Executive Summary & Vision

Sort IT is an institutional-grade financial intelligence platform that eliminates the gap between raw expense data and CFO-level insight. Unlike legacy tools (QuickBooks, Expensify, Zoho Books) that produce ledgers for accountants, Sort IT produces living financial narratives for decision-makers.

PROBLEM STATEMENT

The ₹4,200 Cr Indian SMB accounting software market is dominated by tools designed in the 2000s. They are:

- → Row-based, not insight-based — raw data, zero intelligence
- → Built for compliance, not strategy — tax-first, growth-last
- → Visually hostile — Excel-era UI in a Figma-era world
- → AI-absent — zero natural language querying or anomaly detection

SORT IT'S CORE BET

One breakthrough product insight underpins everything:

> "Finance teams do not need more data. They need financial intelligence delivered with the visual fluency of modern design tools."

- → Replace tables with cinematic data visualization
- → Replace manual reports with AI-generated financial narratives
- → Replace reactive tracking with predictive financial posture

TARGET MARKET (MVP)

Segment	Profile	Pain Point
VC-Backed Startups	10–100 people, Series A/B, monthly investor updates	No clean burn visualization; manual board deck prep
D2C & E-commerce	₹1Cr–50Cr GMV, multi-channel spend	CAC vs revenue attribution is manual and delayed
Founder + HNIs	Running personal + business money together	No single view; uses multiple apps
Professional Services	Consulting, law, architecture firms	Project-level P&L tracking is impossible in Tally

02 Product Philosophy & Design System

DESIGN DNA

Sort IT's visual identity is built on a single conviction: financial data should feel as powerful as it actually is. The aesthetic is not 'fintech' — it is institutional command infrastructure.

Principle	Expression	Anti-pattern Avoided
Terminal Precision	Monospace numbers, tight grid, data density	Airy SaaS whitespace that hides information
Cinematic Data	Animated Sankey flows, live counters, scroll-reveal	Static charts that look like Excel
Surgical Color	#00FF89 as the single signal color on pure black	Gradient-heavy 'modern' fintech palettes
Institutional Weight	Bold type, deep hierarchy, serious tone	Playful icons and hand-drawn elements
Intelligence First	AI response in the same viewport as raw data	Separate 'reports' page away from context

COLOR SYSTEM

Token	Hex	Role	Usage
`--color-signal`	`#00FF89`	Primary Signal	CTAs, active states, data highlights, AI outputs
`--color-signal-dim`	`#00CC6E`	Signal Dim	Hover states, secondary actions
`--color-bg-base`	`#0A0A0A`	Base Background	Page background, modal overlays
`--color-bg-surface`	`#111111`	Surface	Cards, panels, sidebars
`--color-bg-elevated`	`#1A1A1A`	Elevated	Dropdowns, tooltips, popovers
`--color-border`	`#2A2A2A`	Border	Card borders, dividers, input borders
`--color-text-primary`	`#F5F5F5`	Primary Text	Headings, labels, critical data
`--color-text-secondary`	`#9A9A9A`	Secondary Text	Descriptions, meta, placeholder
`--color-danger`	`#FF4444`	Danger	Anomalies, overbudget, alerts

`--color-warning`	`#FFB800`	Warning	Approaching limit, moderate risk
`--color-positive`	`#00FF89`	Positive	Revenue, savings, growth metrics

TYPOGRAPHY SYSTEM

Token	Font	Weight / Size	Use Case
Display	Syne	800 / 48–72px	Page titles, hero numbers, brand marks
Heading	Syne	700 / 24–36px	Section headers, card titles
Label	Space Mono	500 / 11–13px	Data labels, category tags, uppercase caps
Body	Inter	400 / 14–16px	AI narratives, descriptions, tooltips
Numeric	Space Mono	600–700 / 14–32px	All financial numbers, percentages, dates
Code / Query	JetBrains Mono	400 / 12–14px	AI query input, formula fields

03 MVP Feature Set

The MVP is scoped to deliver maximum perceived differentiation with minimum engineering complexity. Every feature either (a) is visually unique vs competitors, or (b) delivers AI intelligence unavailable anywhere else in the Indian market.

IN SCOPE — MVP V1.0

F01 · Authentication & Workspace

- → Email + password signup with email verification
- → Google OAuth via NextAuth.js
- → Single workspace per user (multi-entity in v2)
- → Business profile setup: name, currency (INR/USD), fiscal year
- → Invite team members with role: Admin / Viewer

F02 · Expense Entry & Management

- → Manual expense entry form: amount, vendor, date, category, notes, project tag
- → Bulk CSV import with intelligent column mapping (AI-assisted)
- → Drag-and-drop receipt upload (image/PDF) → OCR extraction via Google Vision API
- → Inline editing of all expense fields
- → Soft-delete with 30-day restore window
- → Recurring expense detection and auto-tagging
- → Multi-currency input with live INR conversion (ExchangeRate API)

F03 · Visual Command Dashboard

- → Live KPI strip: Total Spend, Budget Utilization %, Runway, Anomaly Count
- → 6-month Spend vs Revenue vs Budget animated bar chart (D3.js)
- → Category breakdown with animated horizontal bars + trend delta
- → Cash Runway gauge (SVG arc visualization)
- → Spend Velocity sparklines per category
- → Heatmap calendar: daily spend intensity (Jan–Dec grid)
- → All charts respond to date-range filter: 7D / 30D / 90D / YTD / Custom

F04 · Money Flow Visualization (Differentiator #1)

- → Animated Sankey diagram: income sources → expense categories → net
- → Hover states reveal: exact amount, % of total, MoM delta
- → Flow width proportional to monetary volume
- → Color-coded by category; anomalous flows highlighted in danger red
- → Screenshot-to-share export button (PNG/SVG)

F05 · AI Financial Copilot (Differentiator #2)

- → Natural language query input: 'What drove infra cost up this quarter?'
- → Context-injected prompt: full financial snapshot sent with every query
- → Streaming response with typewriter render effect
- → Pre-built suggestion chips: Top 5 questions surfaced by AI based on current data
- → Anomaly narrative: auto-generated 3-sentence summary of detected issues
- → Scenario modeling: 'If I cut X by 20%, what happens to runway?' answered in prose
- → Powered by Anthropic Claude API (claude-sonnet model)

F06 · Anomaly Detection Engine

- → Duplicate payment detection: same vendor + ±5% amount within 15 days

- → Spend spike alerts: category exceeds 2× rolling 90-day average
- → Vendor-new alert: first-time payment to unknown vendor over ₹25,000
- → Budget breach warning: category hits 80% and 100% of budget
- → All anomalies surfaced in dashboard with dismiss / investigate actions
- → Email digest: weekly anomaly summary sent every Monday 9AM

F07 · Budget Management

- → Per-category monthly budget setting
- → Visual budget health: color-coded progress bars (green/amber/red)
- → Real-time remaining budget counter in command dashboard
- → Historical budget vs actual accuracy tracking

F08 · Transaction Ledger

- → Sortable, filterable transaction table (all dimensions)
- → Advanced filter panel: date range, category, vendor, amount range, project, flag status
- → Inline flag/unflag for anomaly status
- → Bulk categorization: select N transactions → assign category
- → Vendor auto-complete from transaction history

F09 · Reports & Export

- → Monthly Financial Summary: auto-generated PDF with charts + AI narrative
- → Burn Rate Report: 3-month trend with projection
- → Category Breakdown: table + pie + trend
- → CSV export of any filtered transaction view
- → All reports: branded with user's business name, Sort IT design language

F10 · Onboarding Flow

- → 5-step guided setup wizard (animated, skippable)
- → Demo data preloaded for immediate 'wow moment' — full dashboard visible in <60 seconds
- → Guided tour overlay highlighting AI Copilot, Sankey, and anomaly panel

OUT OF SCOPE — V1.0 (ROADMAP V2+)

Feature	Reason Deferred	Target Version
Bank account integration (Plaid/Setu)	API cost + RBI compliance complexity	v2.0
GST filing integration	Regulatory overhead; separate module needed	v2.0
Mobile app (React Native)	Focus web first; achieve PMF	v2.0
Multi-entity / multi-company	Org model complexity	v2.0
Tally / Zoho connector	Custom API for each; post-PMF	v2.1
Payroll tracking	HR data sensitivity; different persona	v3.0
P&L statement generation	Accounting rules complexity	v2.0

Custom AI model fine-tuning	Requires scale dataset	v3.0

04 Technical Architecture

SYSTEM ARCHITECTURE OVERVIEW

Sort IT uses a modern three-tier architecture with a stateless API layer, enabling independent scaling of frontend, backend, and AI services.

```
┌──────────────────────────────────────────────────────────┐
│                    SORT IT ARCHITECTURE                  │
├──────────────────────────────────────────────────────────┤
│  CLIENT LAYER                                            │
│  Next.js 14 (App Router)   ·  Vercel Edge Network        │
│  D3.js Visualizations   ·  Framer Motion   ·  Zustand    │
├──────────────────────────────────────────────────────────┤
│  API LAYER                                               │
│  FastAPI (Python 3.11)   ·  Railway / AWS ECS            │
│  JWT Auth   ·  Redis Cache   ·  Background Jobs (Celery) │
├──────────────────────────────────────────────────────────┤
│  DATA LAYER                                              │
│  PostgreSQL 15 (Supabase)   ·  Redis (Upstash)           │
│  S3-compatible (Cloudflare R2) for receipts              │
├──────────────────────────────────────────────────────────┤
│  EXTERNAL SERVICES                                       │
│  Anthropic Claude API   ·  Google Vision OCR             │
│  Resend (Email)   ·  ExchangeRate API   ·  Sentry        │
└──────────────────────────────────────────────────────────┘
```

TECHNOLOGY STACK — COMPLETE

Layer	Technology	Version	Rationale
Frontend	**Next.js (App Router)**	14.x	SSR + RSC for fast FCP; file-based routing
Frontend	**TypeScript**	5.x	Type safety across full stack
Frontend	**Tailwind CSS**	3.x	Utility-first; dark theme trivial
Frontend	**Framer Motion**	11.x	Production-grade animations; simple API
Frontend	**D3.js**	7.x	Sankey + custom viz; full control
Frontend	**Recharts**	2.x	Fast line/bar charts; Recharts over Chart.js
Frontend	**Zustand**	4.x	Lightweight global state; no Redux overhead
Frontend	**React Query**	5.x	Server state, caching, background refetch
Frontend	**React Hook Form + Zod**	latest	Form handling + runtime validation

Backend	**FastAPI**	`0.110+`	Async-first, auto OpenAPI docs, Python AI libs
Backend	**SQLAlchemy 2.0**	`2.x`	ORM with async support
Backend	**Alembic**	`1.x`	Database migrations
Backend	**Celery + Redis**	`5.x`	Async jobs: OCR, email, anomaly detection
Backend	**Pydantic v2**	`2.x`	Data validation; 10× faster than v1
Database	**PostgreSQL**	`15`	ACID, JSON columns for AI metadata
Database	**Redis (Upstash)**	`7.x`	Session cache, rate limiting, job queue
Storage	**Cloudflare R2**	—	S3-compatible; no egress fees
Auth	**NextAuth.js**	`4.x`	JWT + OAuth; Supabase Auth in v2
AI	**Anthropic Claude API**	`claude-sonnet`	Financial reasoning quality
AI	**Google Vision API**	`v1`	Receipt OCR; 1000 free/month
Email	**Resend**	`latest`	Deliverability + React Email templates
Infra	**Vercel**	—	Frontend; edge functions for API routes
Infra	**Railway**	—	FastAPI backend; auto-deploy from GitHub
Monitoring	**Sentry**	—	Error tracking frontend + backend
Analytics	**PostHog**	—	Product analytics; self-hostable

05 Frontend Engineering Specification

PROJECT STRUCTURE

```
sort-it/
├── app/                          # Next.js App Router
│   ├── (auth)/                   # Auth route group
│   │   ├── login/page.tsx
│   │   └── signup/page.tsx
│   ├── (dashboard)/              # Protected route group
│   │   ├── layout.tsx            # Sidebar + header shell
│   │   ├── page.tsx              # /dashboard (Command Center)
│   │   ├── flows/page.tsx        # Money Flow Sankey
│   │   ├── intelligence/page.tsx # AI Copilot + Insights
│   │   ├── transactions/page.tsx # Ledger
│   │   ├── budgets/page.tsx      # Budget Management
│   │   └── reports/page.tsx      # Report generation
│   ├── api/                      # API route handlers
│   │   ├── auth/[...nextauth]/   # NextAuth endpoints
│   │   └── ai/stream/route.ts   # Streaming AI proxy
│   └── layout.tsx                # Root layout
├── components/
│   ├── charts/                   # D3 + Recharts components
│   │   ├── SankeyFlow.tsx
│   │   ├── SpendBarChart.tsx
│   │   ├── RunwayGauge.tsx
│   │   ├── SparkLine.tsx
│   │   └── HeatmapCalendar.tsx
│   ├── ai/
│   │   ├── CopilotPanel.tsx
│   │   └── InsightCard.tsx
│   ├── ui/                       # Design system primitives
│   │   ├── Button.tsx
│   │   ├── Card.tsx
│   │   ├── Badge.tsx
│   │   ├── Input.tsx
│   │   └── Modal.tsx
│   └── layout/
│       ├── Sidebar.tsx
│       └── CommandHeader.tsx
├── lib/
│   ├── api-client.ts             # Typed API wrapper
│   ├── formatters.ts             # Currency, date formatters
│   └── anomaly.ts               # Client-side detection helpers
├── store/                        # Zustand stores
│   ├── useExpenseStore.ts
│   └── useUIStore.ts
└── types/                        # Shared TypeScript types
```

KEY COMPONENT SPECIFICATIONS

SankeyFlow.tsx — Signature Visualization

→ Built with D3 d3-sankey layout algorithm
→ Animated: nodes fade in at 0ms, links draw from left at 300ms staggered
→ Node hover: tooltip with amount, % of total, MoM delta (Framer Motion scale)
→ Link hover: highlight path from source to target, dim all others
→ Responsive: redraws on container resize via ResizeObserver
→ Export: html-to-image captures canvas as PNG

```
interface SankeyProps {
  nodes: { id: string; label: string; color?: string }[];
  links: { source: string; target: string; value: number }[];
  width: number;
  height: number;
  onNodeClick?: (nodeId: string) => void;
}
```

CopilotPanel.tsx — AI Interface

- → Streaming response via EventSource to /api/ai/stream
- → Typewriter animation: characters rendered at 20ms intervals
- → Suggestion chips auto-generated: 5 chips from /api/ai/suggestions (debounced, refreshes on data change)
- → Query history: last 10 queries stored in localStorage, accessible via ↑/↓ arrow keys
- → Markdown rendering of AI response via react-markdown

```
// Streaming implementation
const streamResponse = async (query: string) => {
  const response = await fetch('/api/ai/stream', {
    method: 'POST',
    body: JSON.stringify({ query, context: financialSnapshot }),
  });
  const reader = response.body!.getReader();
  // Render chunks progressively to UI
};
```

PERFORMANCE TARGETS

Metric	Target	Measurement	Tool
First Contentful Paint (FCP)	**< 1.2s**	P75 across sessions	Vercel Analytics
Largest Contentful Paint (LCP)	**< 2.5s**	P75 across sessions	Lighthouse CI
Cumulative Layout Shift (CLS)	**< 0.05**	All pages	Web Vitals
Dashboard Load (with data)	**< 800ms**	After auth, first render	React Query
AI First Token Response	**< 1.5s**	From submit to first char	Custom timer
Sankey Animation FPS	**≥ 60fps**	On M1 + mid-tier Android	Chrome DevTools
Time-to-Interactive	**< 3.0s**	Full dashboard usable	Lighthouse CI

06 Backend Engineering Specification

FASTAPI PROJECT STRUCTURE

```
backend/
├── app/
│   ├── main.py                  # FastAPI app init, CORS, middleware
│   ├── core/
│   │   ├── config.py            # Settings via pydantic-settings
│   │   ├── security.py          # JWT encode/decode
│   │   └── database.py          # Async SQLAlchemy engine
│   ├── api/v1/
│   │   ├── auth.py              # /auth/register, /auth/login
│   │   ├── expenses.py          # CRUD + bulk operations
│   │   ├── analytics.py         # Aggregations, trends
│   │   ├── ai.py                # Claude proxy + streaming
│   │   ├── budgets.py           # Budget CRUD
│   │   └── reports.py           # PDF report generation
│   ├── models/                  # SQLAlchemy ORM models
│   │   ├── user.py
│   │   ├── expense.py
│   │   └── budget.py
│   ├── schemas/                 # Pydantic request/response
│   ├── services/
│   │   ├── anomaly_service.py   # Detection algorithms
│   │   ├── ai_service.py        # Claude integration
│   │   ├── ocr_service.py       # Google Vision wrapper
│   │   └── analytics_service.py # Aggregation logic
│   └── tasks/                   # Celery background tasks
│       ├── weekly_digest.py
│       └── anomaly_scan.py
├── alembic/                     # DB migrations
├── tests/
└── Dockerfile
```

ANOMALY DETECTION ALGORITHMS

Algorithm 1: Duplicate Payment Detection

```
def detect_duplicates(expenses: List[Expense]) -> List[AnomalyFlag]:
    flagged = []
    for i, exp in enumerate(expenses):
        for j, other in enumerate(expenses[i+1:]):
            days_apart = abs((exp.date - other.date).days)
            amount_diff_pct = abs(exp.amount - other.amount) / exp.amount
            if (exp.vendor_name == other.vendor_name
                and days_apart <= 15
                and amount_diff_pct <= 0.05):
                flagged.append(AnomalyFlag(
                    type='DUPLICATE',
                    severity='HIGH',
                    expense_ids=[exp.id, other.id],
                    message=f'Possible duplicate: {exp.vendor_name}'
                ))
    return flagged
```

Algorithm 2: Spend Spike Detection

```
def detect_spend_spikes(expenses: List[Expense]) -> List[AnomalyFlag]:
    # Group by category, compute 90-day rolling average
    # Flag if current period > 2x rolling average
    category_groups = group_by_category(expenses)
    for category, txns in category_groups.items():
        rolling_avg = compute_rolling_avg(txns, days=90)
        current = sum_last_30_days(txns)
        if current > rolling_avg * 2:
            yield AnomalyFlag(type='SPIKE', category=category, ...)
```

07 AI / Intelligence Layer

CLAUDE API INTEGRATION ARCHITECTURE

Every AI query includes a structured financial context object injected as system prompt. This gives Claude the full business snapshot needed for institutional-quality answers.

SYSTEM PROMPT TEMPLATE

```
SYSTEM PROMPT (injected with every query):

You are Sort IT's AI Financial Copilot — an institutional-grade
financial analyst for {business_name}.

CURRENT FINANCIAL CONTEXT:
- Period: {period}
- Total Spend: {total_spend} ({currency})
- Budget Utilization: {budget_pct}%
- Cash Runway: {runway_months} months
- Active Anomalies: {anomaly_count}

CATEGORY BREAKDOWN:
{category_json}

RECENT ANOMALIES:
{anomaly_summary}

RESPONSE RULES:
→ Be specific: use actual numbers from context
→ Be direct: CFO-level insight, no generic advice
→ Max 4 sentences unless modeling a scenario
→ Flag if data is insufficient for a confident answer
→ Respond in {language} (default: English)
```

AI FEATURES SPECIFICATION

AI Feature	Implementation	Trigger	Latency Target
Natural Language Query	Streaming Claude API call with full context	User types + submits	< 1.5s first token
Anomaly Narrative	Scheduled daily, stored in DB, surfaced in dashboard	Auto (cron)	Pre-generated
Suggestion Chips	Batch call on data change, 5 question suggestions	Data refresh	< 2s
Scenario Modeling	User query parsed for 'if X then Y' pattern; specialized prompt	User query	< 2s
CSV Column Mapping	Claude reads CSV headers, returns mapping JSON	CSV upload	< 1s

Receipt OCR Cleanup	Google Vision extracts text; Claude parses into structured fields	Receipt upload	< 3s
Weekly Digest	Batch analysis of week's data; email narrative generated	Celery cron	Background

08 Database Schema

```
-- USERS
CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email       TEXT UNIQUE NOT NULL,
  name        TEXT,
  password_hash TEXT,
  created_at  TIMESTAMPTZ DEFAULT NOW(),
  updated_at  TIMESTAMPTZ DEFAULT NOW()
);

-- WORKSPACES
CREATE TABLE workspaces (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  owner_id    UUID REFERENCES users(id),
  name        TEXT NOT NULL,
  currency    TEXT DEFAULT 'INR',
  fiscal_year_start INT DEFAULT 4,   -- April for India
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- EXPENSES (core table)
CREATE TABLE expenses (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id    UUID REFERENCES workspaces(id),
  amount          NUMERIC(15,2) NOT NULL,
  currency        TEXT DEFAULT 'INR',
  amount_inr      NUMERIC(15,2),         -- converted amount
  vendor_name     TEXT NOT NULL,
  category        TEXT NOT NULL,
  sub_category    TEXT,
  date            DATE NOT NULL,
  notes           TEXT,
  project_tag     TEXT,
  receipt_url     TEXT,
  is_recurring    BOOLEAN DEFAULT FALSE,
  recurrence_id   UUID,
  is_deleted      BOOLEAN DEFAULT FALSE,
  created_by      UUID REFERENCES users(id),
  ai_metadata     JSONB,                 -- OCR confidence, AI tags
  created_at      TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_expenses_workspace_date ON expenses(workspace_id, date);
CREATE INDEX idx_expenses_category ON expenses(workspace_id, category);

-- BUDGETS
CREATE TABLE budgets (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id    UUID REFERENCES workspaces(id),
  category        TEXT NOT NULL,
  month           DATE NOT NULL,   -- first day of month
  amount          NUMERIC(15,2) NOT NULL,
  UNIQUE(workspace_id, category, month)
);

-- ANOMALIES
CREATE TABLE anomaly_flags (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id    UUID REFERENCES workspaces(id),
  type            TEXT NOT NULL,  -- DUPLICATE|SPIKE|NEW_VENDOR|BUDGET_BREACH
  severity        TEXT NOT NULL,  -- HIGH|MEDIUM|LOW
  expense_ids     UUID[],
  message         TEXT NOT NULL,
  is_dismissed    BOOLEAN DEFAULT FALSE,
  detected_at     TIMESTAMPTZ DEFAULT NOW()
);
```

```database
-- AI QUERY HISTORY
CREATE TABLE ai_queries (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id    UUID REFERENCES workspaces(id),
  user_id         UUID REFERENCES users(id),
  query           TEXT NOT NULL,
  response        TEXT,
  tokens_used     INT,
  created_at      TIMESTAMPTZ DEFAULT NOW()
);
```

09 API Design

REST ENDPOINTS — COMPLETE

Method	Endpoint	Auth	Description
POST	/api/v1/auth/register	Public	Create account
POST	/api/v1/auth/login	Public	Return JWT + refresh token
POST	/api/v1/auth/refresh	Refresh Token	New access token
GET	/api/v1/workspace	JWT	Get workspace details
PUT	/api/v1/workspace	JWT	Update workspace settings
GET	/api/v1/expenses	JWT	Paginated expense list + filters
POST	/api/v1/expenses	JWT	Create single expense
PUT	/api/v1/expenses/{id}	JWT	Update expense
DELETE	/api/v1/expenses/{id}	JWT	Soft delete expense
POST	/api/v1/expenses/bulk	JWT	Bulk create from CSV parse
POST	/api/v1/expenses/upload-receipt	JWT	Upload + OCR receipt
GET	/api/v1/analytics/summary	JWT	Dashboard KPIs
GET	/api/v1/analytics/trends	JWT	Monthly trend data
GET	/api/v1/analytics/categories	JWT	Category breakdown
GET	/api/v1/analytics/sankey	JWT	Sankey nodes + links data
GET	/api/v1/budgets	JWT	All budgets for workspace
POST	/api/v1/budgets	JWT	Create/update budget
GET	/api/v1/anomalies	JWT	Active anomaly flags
PUT	/api/v1/anomalies/{id}/dismiss	JWT	Dismiss an anomaly
POST	/api/v1/ai/query	JWT	Synchronous AI query
POST	/api/v1/ai/stream	JWT	SSE streaming AI query
GET	/api/v1/ai/suggestions	JWT	5 suggested questions
POST	/api/v1/reports/generate	JWT	Trigger PDF report
GET	/api/v1/reports/{id}	JWT	Download generated report

10 Security & Compliance

SECURITY ARCHITECTURE

→ JWT access tokens (15-min expiry) + HTTP-only cookie refresh tokens (7-day)
→ Bcrypt password hashing (cost factor 12)
→ Row-level security: all DB queries scoped to workspace_id from JWT claim
→ Rate limiting: 100 req/min per IP (Redis sliding window) via FastAPI middleware
→ CORS: strict whitelist — only sortit.finance and localhost in dev
→ All API communication over TLS 1.3 only
→ Receipts in Cloudflare R2 with signed URLs (1-hour expiry)
→ Anthropic API key stored in environment secrets only — never exposed to frontend
→ Input sanitization via Pydantic v2 validators on all endpoints
→ SQL injection impossible: SQLAlchemy ORM with parameterized queries only

DATA PRIVACY

→ Financial data encrypted at rest in PostgreSQL (Supabase AES-256)
→ PII fields (email, name) stored separately from transaction data
→ AI queries: financial context sent to Anthropic API — users must accept Terms explicitly
→ Data deletion: hard delete available within 30 days of soft delete
→ GDPR-ready: data export endpoint, right-to-erasure flow built for MVP
→ No third-party analytics scripts with access to financial data (PostHog is self-hostable)

11 UI/UX Design System

PAGE LAYOUTS & NAVIGATION

Global Shell

- → Left sidebar (240px fixed): logo, nav items, workspace switcher, user avatar
- → Top command bar (52px): current page title, date range picker, search, notifications
- → Main content area: fluid, max-width 1440px, centered
- → Sidebar collapses to 56px icon rail on <1200px screens

Navigation Structure

- → Command Center — primary dashboard (default route)
- → Money Flows — Sankey visualization
- → AI Intelligence — Copilot + insights
- → Transactions — full ledger
- → Budgets — budget management
- → Reports — generation + history
- → Settings — workspace, team, integrations

ANIMATION SYSTEM

Element	Animation	Duration	Easing
Page entry	Staggered fade-up (each section +80ms)	400ms	`easeOut`
KPI counters	Count from 0 to value on mount	1200ms	`easeOut`
Progress bars	Width 0 → value on mount	800ms	`spring(stiffness:100)`
Sankey links	Path drawOn from left	600ms staggered	`easeInOut`
AI typewriter	Character-by-character render	20ms/char	`linear`
Card hover	Y -2px, border-color → green	150ms	`easeOut`
Sidebar nav	Active indicator slide	200ms	`spring`
Modal entry	Scale 0.96→1 + fade	200ms	`easeOut`
Anomaly badge	Pulse glow on red	2s infinite	`ease-in-out`
Number update	Flash #00FF89 then settle	400ms	`easeOut`

12 10-Week Build Roadmap

Engineered for a solo developer working 2–3 hours on weekdays and 6–8 hours on weekends. Roughly 25 productive hours/week.

Wk	Phase	Deliverables	Success Criteria
01	**Foundation**	Next.js project setup, Tailwind dark theme, design tokens (CSS vars), font loading, Sidebar + Shell layout, Storybook for components	Design system renders correctly; shell navigates between empty pages
02	**Auth + DB**	NextAuth.js email + Google OAuth, PostgreSQL schema + Alembic migrations, JWT middleware, workspace creation flow, Supabase connection	User can sign up, log in, create workspace
03	**Expense CRUD**	Manual expense entry form, expense list page, edit/delete, categories, Zod validation, React Query integration	Can add, view, edit, delete expenses end-to-end
04	**Dashboard KPIs**	Analytics API endpoints, KPI strip with animated counters, 6-month bar chart (Recharts), category bars with trends, date range filter	Dashboard renders live data from DB with animations
05	**Sankey + Flows**	d3-sankey implementation, animated link drawing, hover tooltips, responsive resize, flows page layout	Sankey renders correctly with real data; 60fps animation
06	**AI Copilot**	Anthropic API integration, streaming SSE endpoint, CopilotPanel UI with typewriter, suggestion chips, query history	AI responds with accurate financial context in <2s first token
07	**Anomaly Engine**	Duplicate detection algorithm, spend spike detection, anomaly flags DB + API, anomaly panel in dashboard, email alerts via Resend	Anomalies correctly detected on sample data; email sends
08	**CSV + OCR**	CSV import with AI column mapping, bulk upload UI, Google Vision OCR integration, receipt upload + parse flow	CSV with 100 rows imports correctly; receipt parsed to form
09	**Reports + Budget**	Budget CRUD + visual health bars, PDF report generation (ReportLab), report history page, weekly digest email	Budget tracks correctly; PDF downloads with correct data
10	**Polish + Launch**	Onboarding wizard, demo data seeder, Lighthouse audit + performance fixes, Sentry + PostHog setup, production deploy, landing page	LCP <2.5s, all core flows work end-to-end in production

13 Infrastructure & DevOps

DEPLOYMENT ARCHITECTURE

Service	Platform	Cost (MVP)	Scaling Path
Frontend	Vercel (Hobby→Pro)	₹0→₹1,700/mo	Auto-scales; upgrade on traction
Backend API	Railway Starter	₹420/mo	Scale to Railway Team; later ECS Fargate
PostgreSQL	Supabase Free→Pro	₹0→₹2,000/mo	Managed; migrate to RDS at scale
Redis	Upstash Free	₹0	Pay-per-request; scales naturally
File Storage	Cloudflare R2	₹0 (10GB free)	No egress fees; ideal for receipts
Email	Resend Free	₹0 (100/day free)	Upgrade to $20/mo at growth
Monitoring	Sentry Free	₹0	Upgrade at team size
Analytics	PostHog Free	₹0	1M events/month free

CI/CD PIPELINE

```
GitHub Actions Workflow:

On PR → main:
  1. Run TypeScript type check (tsc --noEmit)
  2. Run ESLint
  3. Run Pytest (backend unit tests)
  4. Run Playwright E2E tests (auth flow + expense CRUD)
  5. Lighthouse CI (LCP, CLS, FCP gates)

On merge to main:
  1. Deploy frontend to Vercel (automatic)
  2. Run Alembic migrations on Railway PostgreSQL
  3. Deploy FastAPI to Railway (Docker)
  4. Notify Sentry of new release

Environment: dev | staging | production
Secrets: Doppler (secrets management, syncs to Vercel/Railway)
```

14 Success Metrics & KPIs

LAUNCH SUCCESS CRITERIA (WEEK 10)

Metric	MVP Target	Stretch	Measurement
Beta Signups (Day 30)	**50 users**	200 users	PostHog
Paid Conversions (Day 60)	**5 paying users**	20 users	Stripe Dashboard
Weekly Active Users	**40% of signups**	60% of signups	PostHog retention
AI Queries per User/Week	**>3 queries**	>8 queries	ai_queries table
Expense Entries (avg user)	**>20/month**	>50/month	DB aggregate
Product Hunt Upvotes	**100+**	500+	PH dashboard
Core Web Vitals	**LCP <2.5s, CLS <0.05**	LCP <1.5s	Lighthouse CI
Support Tickets	**<5/week**	<2/week	Email inbox
MRR at Day 90	**₹12,500**	₹50,000	Stripe

NORTH STAR METRIC

> "Weekly AI Copilot Queries per Active Workspace — the single metric that captures whether Sort IT has become indispensable."

If users query the AI more than 3x/week, they have internalized Sort IT as a decision-making tool, not just a tracking tool. That is the product-market fit signal. Build everything to drive this number.

SORT IT — MVP Technical Specification v1.0
Confidential — April 2026